This article was downloaded by: [Universita di Padova], [L. De Giovanni]
On: 14 May 2013, At: 09:58
Publisher: Taylor & Francis
Informa Ltd Registered in England and Wales Registered Number: 1072954 Registered office: Mortimer House, 37-41 Mortimer Street, London W1T 3JH, UK



International Journal of Production Research
Publication details, including instructions for authors and subscription information:
http://www.tandfonline.com/loi/tprs20

An adaptive genetic algorithm for large-size open stack problems

L. De Giovanni [a], G. Massi [b] & F. Pezzella [b]

[a] Dipartimento di Matematica, Università degli Studi di Padova, via Trieste 63, 35121 Padova, Italy

[b] Dipartimento di Ingegneria dell'Informazione, Università Politecnica delle Marche, via Brecce Bianche, 60131 Ancona, Italy

Published online: 23 Mar 2012.

To cite this article: L. De Giovanni , G. Massi & F. Pezzella (2013): An adaptive genetic algorithm for large-size open stack problems, International Journal of Production Research, 51:3, 682-697

To link to this article: http://dx.doi.org/10.1080/00207543.2012.657256

PLEASE SCROLL DOWN FOR ARTICLE

Full terms and conditions of use: http://www.tandfonline.com/page/terms-and-conditions

This article may be used for research, teaching, and private study purposes. Any substantial or systematic reproduction, redistribution, reselling, loan, sub-licensing, systematic supply, or distribution in any form to anyone is expressly forbidden.

The publisher does not give any warranty express or implied or make any representation that the contents will be complete or accurate or up to date. The accuracy of any instructions, formulae, and drug doses should be independently verified with primary sources. The publisher shall not be liable for any loss, actions, claims, proceedings, demand, or costs or damages whatsoever or howsoever caused arising directly or indirectly in connection with or arising out of the use of this material.

International Journal of Production Research
Vol. 51, No. 3, 1 February 2013, 682–697

Taylor & Francis
Taylor & Francis Group

An adaptive genetic algorithm for large-size open stack problems

L. De Giovanni[a]*, G. Massi[b] and F. Pezzella[b]

[a]*Dipartimento di Matematica, Università degli Studi di Padova, via Trieste 63, 35121 Padova, Italy;* [b]*Dipartimento di Ingegneria dell'Informazione, Università Politecnica delle Marche, via Brecce Bianche, 60131 Ancona, Italy*

(*Received 6 April 2011; final version received 1 January 2012*)

The problem of minimising the maximum number of open stacks arises in many contexts (production planning, cutting environments, very-large-scale-integration circuit design, etc.) and consists of finding a sequence of tasks (products, cutting patterns, circuit gates, etc.) that determines an efficient utilisation of resources (stacks). We propose a genetic approach that combines classical genetic operators (selection, order crossover and pairwise interchange mutation) with an adaptive search strategy, where intensification and diversification phases are obtained by neighbourhood search and by a composite and dynamic fitness function that suitably modifies the search landscape. Computational tests on random and real-world benchmarks show that the proposed approach is competitive with the state of the art for large-size problems, providing better results for some classes of instances.

Keywords: combinatorial optimisation; genetic algorithms; production planning; sequencing; minimisation of open stacks

Downloaded by [Universita di Padova], [L. De Giovanni] at 09:58 14 May 2013

1. Introduction

The Minimisation of Open Stacks Problem generalises a wide class of problems arising in many contexts. For explanatory purposes, we refer to cutting stock environments, where saw machines cut large panels into smaller pieces (items). Each panel is cut according to one among many cutting patterns. A cutting pattern may contain items of different types. When a panel is cut, the produced items are heaped on stacks around the saw machine. Each stack collects items of the same type, that is items having the same size and/or to be processed together in the following production phases. When the production of an item type starts, a new stack is opened. A stack is closed when the production of all the items of the same type ends (we assume a large enough stack capacity). As a consequence, a stack remains open from the first time the related item type is cut, until the last item of that type is produced: as items of the same type may be spread among more than one cutting pattern, a stack may stay open even if no items of that type are currently output by the machine. The number of simultaneously open stacks is one of the most critical indicators of the efficiency of the cutting process. In fact, plants have limited available positions for stacks: each time another stack is required, it has to be opened far away from the machine or, alternatively, a busy position has to be made available by moving items to and from a temporary warehouse. These operations imply additional handling and human intervention, which are among the most relevant cutting process bottlenecks, leading to time wasting and possible damage. Hence, stack queues as small as possible are desirable. The Minimisation of Open Stacks Problem (MOSP) considers a given set of cutting patterns and determines the cutting sequence that minimises the maximum number of simultaneously open stacks.

Any MOSP instance can be described by the number of item types m, the number of patterns n, and by an $m \times n$ matrix M, where $M(i,j)$ is equal to 1 if pattern j contains any item of type i, and 0 otherwise. A possible mathematical formulation for MOSP is the Mixed Integer Linear Programming (MILP) model proposed by Baptiste (2005) and reported in Figure 1. The cutting sequence is defined by binary decision variables x_{jk} taking value 1 if pattern j is scheduled in position k, and 0 otherwise: constraints (3) and (4) force each pattern to be cut in one and only one position. Three sets of binary variables are used to count the number of open stacks needed at each position k of the cutting sequence: variables s_{ik}, equal to one from the first position item type i is in process on (constraints (6)); e_{ik}, equal to one from position 1 until the production of i ends (constraints (7)); o_{ik}, set by constraints (8) to one if the stack for item type i is open when the kth pattern is in process. Constraints (5) force o_{ik}

*Corresponding author. Email: luigi@math.unipd.it

ISSN 0020–7543 print/ISSN 1366–588X online
© 2013 Taylor & Francis
http://dx.doi.org/10.1080/00207543.2012.657256
http://www.tandfonline.com

$$\min \quad y \tag{1}$$

$$\text{s.t.} \quad \sum_{i=1}^{m} o_{ik} \leq y \qquad \forall k = 1, \ldots, n \tag{2}$$

$$\sum_{j=1}^{n} x_{jk} = 1 \qquad \forall k = 1, \ldots, n \tag{3}$$

$$\sum_{k=1}^{n} x_{jk} = 1 \qquad \forall j = 1, \ldots, n \tag{4}$$

$$x_{jk} \leq o_{ik} \qquad \forall i = 1, \ldots, m,\ j = 1, \ldots, n : M(i,j) = 1,\ k = 1, \ldots, n \tag{5}$$

$$s_{ik} \geq s_{i\,k-1} \qquad \forall i = 1, \ldots, m,\ k = 2, \ldots, n \tag{6}$$

$$e_{ik} \leq e_{i\,k-1} \qquad \forall i = 1, \ldots, m,\ k = 2, \ldots, n \tag{7}$$

$$o_{ik} = s_{ik} + e_{ik} - 1 \qquad \forall i = 1, \ldots, m,\ k = 1, \ldots, n \tag{8}$$

$$x_{jk} \in \{0, 1\} \qquad \forall j = 1, \ldots, n,\ k = 1, \ldots, n \tag{9}$$

$$s_{ik}, e_{ik}, o_{ik} \in \{0, 1\} \qquad \forall i = 1, \ldots, m,\ k = 1, \ldots, n \tag{10}$$

$$y \in \mathbb{R} \tag{11}$$

Figure 1. A mathematical formulation for MOSP (Baptiste 2005).

to 1 if the pattern j is processed in position k and contains i, and, by (6), (7) and (8), the stack remains open from the first to the last time i is cut. Constraints (2) count the number of open stacks at each position k and set variable y equal to the maximum number of simultaneously open stacks, minimised by the objective function (1).

MOSP is $\mathcal{NP}$-hard and can be used to model problems in different application contexts (Linhares and Yanasse 2002). For example, in flexible manufacturing environments, MOSP models the minimisation of open clients' orders, to limit logistical overhead (inventory costs, split deliveries, lead times, etc.). In Very Large Scale Integration (VLSI) circuit design, MOSP models the Gate Matrix Layout Problem (GMLP), where subsets of vertical gates have to be interconnected by horizontal wires. Non-overlapping wires can lay on the same horizontal level. The number of horizontal levels determines the circuit area and depends on how gates are sorted, and can be minimised by solving a MOSP.

2. Previous work

The literature on MOSP is rich and has been increasing in recent years. The earliest works mostly propose heuristics (Yuen 1995), neighbourhood search (Andreatta *et al*. 1989, Faggioli and Bentivoglio 1998, Fink and Voß 1999) and branch-and-bound (Yuen and Richardson 1995). More recently, advanced evolutionary approaches have been devised and tested on literature benchmarks, including the largest available MOSP instances: the Constructive Genetic Algorithm, the Population Training Algorithm (de Oliveira and Lorena 2002, 2005) and the Parallel Memetic Algorithm (Mendes and Linhares 2004). They all integrate basic genetic operators with a 2-OPT local search and use different methods to preserve solution diversification, respectively: including schemata (incomplete sequences to be filled-in by a constructive heuristic) in the population, a training heuristic towards diversified target solutions, and multiple populations. de Oliveira and Lorena (2006) use the clustering search paradigm to implement two algorithms for MOSP, where a genetic approach (Evolutionary Clustering Search) or a randomised greedy procedure (Greedy Randomised Adaptive Clustering Search) generate solutions to be iteratively clustered by an assimilation process (diversification) and exploited by local search (intensification). MILP models are also available to solve small/medium-size instances to optimality, based on fixing the pattern schedule (Baptiste 2005), on the relation between MOSP and consecutive-ones matrices (De Giovanni *et al*. 2011) or on perfect elimination schemes in interval graphs (Lopes and Valério de Carvalho 2010). A fast exact algorithm for MOSP has been proposed by Garcia de la Banda and Stuckey (2007): it exploits dynamic programming to reduce the search space and uses branch-and-bound to repeatedly solve the decision version of MOSP, where the number of available stacks is fixed. The approach has been improved by Chu and Stuckey (2009), providing the state-of-the-art algorithm for MOSP. The key feature is changing the branching strategy according to Becceneri *et al*. (2004), so that item types sequences

Downloaded by [Universita di Padova], [L. De Giovanni] at 09:58 14 May 2013

(a) Production matrix P

$t \setminus p$	1	2	3	4	5	6
A	2	0	0	2	3	0
B	0	0	0	1	1	0
C	0	1	0	1	0	0
D	1	1	0	2	0	2
E	0	0	3	0	0	1
F	0	2	0	0	1	0
G	0	1	1	0	0	0
H	0	0	2	0	0	2
I	3	3	0	0	0	0

(b) 0/1 Production matrix M

$t \setminus p$	1	2	3	4	5	6
A	1	0	0	1	1	0
B	0	0	0	1	1	0
C	0	1	0	1	0	0
D	1	1	0	1	0	1
E	0	0	1	0	0	1
F	0	1	0	0	1	0
G	0	1	1	0	0	0
H	0	0	1	0	0	1
I	1	1	0	0	0	0

Figure 2. A sample instance and related production matrices.

Downloaded by [Universita di Padova], [L. De Giovanni] at 09:58 14 May 2013

are searched, instead of pattern sequences: new dominance rules and nogood recording can be defined, able to limit the required computational resources. Further, a fast heuristic is introduced, as well as a relaxation method to identify core sub-problems, which allows the algorithm of Chu and Stuckey (2009) to very efficiently solve instances of up to 125 patterns and item types.

The scope of this work is a new optimisation procedure for MOSP with a large number of patterns and/or item types, based on a genetic approach. In order to address the peculiarities of the MOSP search space, a new fitness function is proposed, combining different performance indicators, namely the maximum number of open stacks and other indicators related to both global and, differently from previous genetic approaches, local features of the pattern sequences: these indicators are described in Section 3. In order to guarantee a sufficient exploration of the solution space, we propose an algorithm that alternates between intensification and diversification search phases. Instead of introducing further algorithmic components, as in the state-of-the-art metaheuristics cited above, we exploit the definition of the fitness function: the weights of the different components are dynamically adapted to suitably modify the search landscape and either concentrate the search in the current solutions' region (intensification), or move towards solutions with different features (diversification). Local search is also used to improve over the best individuals, trying to trade-off the effectiveness in finding good solutions and the time-efficiency of the algorithm. The general schema of the proposed optimisation procedure and a description of its components is reported in Section 4. The results of extensive computational tests on benchmarks from the literature and industry and on a new set of instances are shown in Section 5. Finally, Section 6 gives some concluding remarks and further research directions.

3. Pattern sequencing performance indicators

In this section, we define the performance indicators used by the proposed algorithm for MOSP, namely:

(1) the Maximum number of simultaneously Open Stacks (MOS);
(2) the Time of Open Stacks (TOS);
(3) the maximum Increment in the number of Open Stacks (IOS); and
(4) the number of new item types (NEW).

To provide examples, we consider the instance of Figure 2, with nine different item types ($t = \mathrm{A}, \ldots, \mathrm{I}$) and six cutting patterns ($p = 1, \ldots, 6$). The instance is described by the *Production Matrix P* (Figure 2(a)). P contains a row for each item type t and a column for each pattern p: position $P(t, p)$ reports the number of items of type t contained in pattern p. As we consider stacks of unlimited capacity, we can use the 0/1 *Production Matrix M*, where $M(t, p) = 1$ if pattern p contains item type t, and 0 otherwise (Figure 2(b)).

MOS is the objective function of MOSP and can be easily computed starting from M. First, we consider the columns in the order of the given pattern sequence σ; then, all the '0' between two '1' in the same row are switched to 1. The obtained matrix is called the *Stack Matrix Q*: $Q(t, p)$ is 1 if the stack for item type t is opened at the time pattern p is processed, and 0 otherwise (Q relates to variables o in the MILP model of Figure 1). For the above example, Figure 3 reports the stack matrices (switched elements in italics) and the performance indicators of two different sequences: $\sigma_1 = \langle 1, 2, 3, 4, 5, 6 \rangle$ (Figure 3(a)) and $\sigma_2 = \langle 4, 5, 1, 2, 6, 3 \rangle$ (Figure 3(b)).

MOS can be obtained by summing up the entries in the same column and taking the maximum over all the columns. If Q_σ is the stack matrix related to the pattern sequence σ, m the number of item types and n the number of

(a) Stack matrix of the original sequence

Sequence $\sigma_1 = \langle 1, 2, 3, 4, 5, 6 \rangle$

$t \setminus p$	1	2	3	4	5	6	
A	1	*1*	*1*	1	1	0	
B	0	0	0	1	1	0	
C	0	1	*1*	1	0	0	
D	1	1	*1*	1	*1*	1	
E	0	0	1	*1*	*1*	1	
F	0	1	*1*	*1*	1	0	
G	0	1	1	0	0	0	
H	0	0	1	*1*	*1*	1	
I	1	1	0	0	0	0	
MOS	3	6	7	7	6	3	**7**
TOS	3	6	7	7	6	3	**32**
NEW	-	3	2	4	1	3	**13**
IOS	-	3	1	0	0	0	**3**

(b) A MOS-optimal solution

Sequence $\sigma_2 = \langle 4, 5, 1, 2, 6, 3 \rangle$

$t \setminus p$	4	5	1	2	6	3	
A	1	1	1	0	0	0	
B	1	1	0	0	0	0	
C	1	*1*	*1*	1	0	0	
D	1	*1*	1	1	1	0	
E	0	0	0	0	1	1	
F	0	1	*1*	1	0	0	
G	0	0	0	1	*1*	1	
H	0	0	0	0	1	1	
I	0	0	1	1	0	0	
MOS	4	5	5	5	4	3	**5**
TOS	4	5	5	5	4	3	**26**
NEW	-	1	2	3	2	1	**9**
IOS	-	1	0	0	0	0	**1**

Figure 3. Stack matrices and pattern-sequencing performance indicators.

patterns, we have

$$MOS(\sigma) = \max_{j=1,\ldots,n} \sum_{i=1}^{m} Q_\sigma(i,j).$$

Looking at Figure 3(a), we obtain $\mathrm{MOS}(\sigma_1) = 7$. MOS changes with pattern sequence: from Figure 3(b), we have $\mathrm{MOS}(\sigma_2) = 5$. Note that this last value cannot be further reduced and σ_2 is a MOS-optimal solution: as pattern 2 contains five item types, at least five stacks are required simultaneously.

Given a set of patterns, many different sequences may lead to the same MOS. This is known as *permutative redundancy*: MOS is related to critical sub-sequences and it is unlikely to change under slight sequence modifications. It is thus important to introduce further performance indicators, with the aim of discriminating, among the sequences with the same or similar MOS, those that are more likely to lie in promising regions of the search space, or those carrying some interesting features to be maintained. To this end, de Oliveira and Lorena (2002) proposed combining MOS and TOS.

TOS corresponds to the total time stacks remaining open and it is related to the dispersion of item types over time. It is obtained by summing up the lengths of the time intervals (measured in number of patterns) to produce each item type and gives a measure of the total utilisation of stacks. In relation to the Stack Matrix Q_σ, we define

$$TOS(\sigma) = \sum_{i=1}^{m} \sum_{j=1}^{n} Q_\sigma(i,j).$$

The minimisation of TOS gives rise to a further pattern sequencing problem known as Minimisation of Order Spread (MORP, Fink and Voß 1999, among others). For example, in GMLP, TOS is the total length of the horizontal wires used to interconnect gates.

MOS and TOS are *global* performance indicators, as they consider the whole pattern sequence: taking just MOS and TOS into account may prevent exploring solutions with relatively poor global performance, but good 'local' features related to partial subsequences or partial precedence relations between patterns. For this reason, further *local* performance indicators are needed to point out sequences leading to interesting regions of the MOSP search space: it is worth exploring (for example, by taking them into the population in a genetic framework) sequences with good IOS and NEW, because they preserve features that are desirable in MOS-optimal solutions. In this paper, two new indicators are introduced, IOS and NEW: they take relations between close (consecutive) patterns into account and have been introduced as local counterparts to MOS and TOS, respectively.

Given a sequence, IOS measures the maximum local increment in the number of open stacks when switching from one pattern to the following one. It is introduced to point out sequences with more *regular* utilisation of stacks, penalising sudden local peaks of MOS due to critical points in the sequence, where many new stacks are opened. IOS can be computed with reference to the stack matrix Q: if $MOS(\sigma, j) = \sum_{i=1}^{m} Q_\sigma(i,j)$ is the number of stacks needed when processing the jth pattern of sequence σ, we have

$$IOS(\sigma) = \max_{j=2,\ldots,n} \{0, MOS(\sigma, j) - MOS(\sigma, j-1)\}.$$

Downloaded by [Universita di Padova], [L. De Giovanni] at 09:58 14 May 2013

(1) *Pre-process* the set of patterns
(2) Encode the population of the *initial generation*
(3) Initialise the *fitness* evaluation criteria
(4) **Repeat**
(5) **Repeat**
(6) *Select* individuals from the current population
(7) Apply *order crossover* to generate offspring
(8) **Until** individuals for *generational replacement* are generated
(9) Copy *elite* and *steady* individuals
(10) **For each** individual **do**
(11) Apply swap *mutation* with a given probability
(12) Evaluate the *fitness*
(13) *Refine* the fittest individuals by *fast local search*
(14) **If** the best individual is unchanged **then**
(15) *Adapt* the mutation probability
(16) **If** current generation corresponds to a *search phase* hop **then**
(17) *Adapt* criteria for *fitness* evaluation
(18) **Until** termination conditions are satisfied
(19) *Post-process* the best individual found during the overall evolution
(20) **Return** the best sequence found.

Figure 4. The Adaptive Genetic Algorithm (AGA).

Downloaded by [Universita di Padova], [L. De Giovanni] at 09:58 14 May 2013

NEW counts the number of item types in each pattern that are not contained in the previous one. The indicator is obtained by summing up, for each pair of consecutive patterns (j, k), the number of item types in k not contained in j. NEW is introduced to privilege sequences with more *similar* consecutive patterns, as this is related, in a local perspective, to smaller item type dispersion: the larger NEW, the more new stacks are likely to be opened, or old stacks remain opened waiting for items that will be produced later. Note that this indicator is different from that cited in the literature (e.g. Yuen 1995), which compares each pattern in position j with the status of the stacks at position $j-1$, and not the pattern in position $j-1$: according to our definition, NEW considers local relations between pairs of consecutive patterns, and not the relation of one pattern with the whole previous subsequence. In fact, NEW can be computed with reference to the original 0/1 production matrix M (elements in italics in Figure 3 must be treated as 0): denoting by M_σ the 0/1 production matrix with the columns ordered according to the pattern sequence σ, we have

$$NEW(\sigma) = \sum_{i=1}^{m} \sum_{j=2}^{n} M_\sigma(i,j) \cdot (M_\sigma(i,j) - M_\sigma(i,j-1)).$$

A combination of both the global and local performance indicators described above (and exemplified in Figure 3) has been shown experimentally to be able to identify promising regions of the MOSP solution space.

4. The Adaptive Genetic Algorithm

The Adaptive Genetic Algorithm (AGA) proposed for MOSP is based on a hybrid genetic–local search approach with learning elitism and a special scheme for adapting its parameters to the MOSP search landscape. As for general evolutionary approaches, AGA iteratively evolves an initial population of solutions according to the principle of 'natural selection': the fittest individuals (chromosomes) are more likely to take part in the reproduction process and transmit their features (genes) to new generations. The basic components of AGA (solution encoding, crossover, mutation and local search) are mostly borrowed from the literature on the Travelling Salesman Problem (TSP), whose combinatorial structure is very similar to that of MOSP: the best solution has to be found among the permutations of n elements (patterns or cities). The general schema of AGA is given in Figure 4: with respect to general genetic algorithms, AGA includes further features (indicated by shaded text) that have been devised and experimentally set to address MOSP peculiarities, in particular:

- a special *fitness function* definition, combining not only global performance indicators (as in some previous genetic approaches), but also the new local indicators introduced above (Step (12));
- a strategy to alternate between *intensification and diversification search phases* through search landscape modifications, obtained by adapting the fitness evaluation criteria (Steps (3), (16) and (17));

- a *refinement operator* based on *fast* local search (Step (13)).

The main components of AGA are detailed in the following.

4.1 *Solution representation*

Before a genetic algorithm can be run, a suitable solution encoding must be devised: the features of a solution are encoded as a set of parameters (genes) to be joined in a string of values (chromosome) representing a solution (individual). Following the classical path representation for the TSP, each gene corresponds to a different pattern and a chromosome is an ordered sequence of n genes, n being the number of patterns: the gene in position j reports the jth pattern to be processed.

4.2 *Pre-processing and initial population*

Several pre-processing steps are available to reduce the size of MOSP instances (Yanasse and Senne 2010). AGA takes the following actions (Step(1) in Figure 4): remove each pattern i contained in another pattern j (MOS remains unchanged if i is scheduled immediately after j); remove all patterns containing a single item type not contained elsewhere (to be sequenced before or after all others). Further, a lower bound is evaluated as the maximum over the measures provided by Miller (2005), including the maximum number of different item types per pattern, and the result of the *Arc Contraction* heuristic (Becceneri *et al*. 2004). The lower bound represents a proven limit to the MOS of any pattern sequence (including the optimal ones), so that the optimal MOS will be greater than or equal to the lower bound. In particular, if a sequence is generated having a MOS equal to the lower bound, it is an optimal sequence.

After pre-processing, AGA initialises the population with a number of individuals given by the parameter π_N (Step (2)). Part of the initial population is obtained from fast constructive heuristics: that proposed by Faggioli and Bentivoglio (1998) and *Heuristic* 3 of Yuen (1995). They start from a pattern and choose the next pattern in the sequence according to greedy criteria, aiming at making the number of open stacks as small as possible. Applying the two heuristics starting from each pattern, different solutions are obtained (twice the number of patterns). These solutions, independently from the generating heuristic, are sorted in lexicographic MOS–TOS order and the best ones, whose number is determined by the parameter π_H, are chosen as starting points for the local search operator described below: we obtain π_H local minima that take part in the initial population. The remaining chromosomes are generated at random.

4.3 *Adaptive fitness function and intensification/diversification*

A straightforward fitness function is given by the objective function, in our case the MOS. In so doing, tests show that, after a number of generations, many of the individuals have the same MOS, even if related sequences may be very different from each other. This is not surprising: due to MOS permutative redundancy, the same value can be obtained by different simultaneously in-progress item types or different critical sub-sequences, so that genetic or local search operators may not lead to MOS changes, even after many perturbations. In terms of search space, if we use MOS to evaluate sequences, several 'plateaus' (regions of close solutions with the same MOS) appear in the search landscape, which makes it difficult to find the way to improve sequences. We thus define a different fitness function to mould the search landscape according to the actual or potential quality of the individuals in the same plateau, so that a descent appears for sequences having the same MOS as their neighbourhood, and further features that are most likely to lead to improved MOS in the following search steps. de Oliveira and Lorena (2002) use a weighted sum of MOS and TOS, the weights implying a lexicographic order of MOS and TOS. In our computational experience, this is not sufficient and we propose the following fitness function (to be minimised), combining both global and local performance indicators:

$$fitness(\sigma, \tau) = \sum_{k \in \{MOS, TOS, IOS, NEW\}} w_k^\tau f_k(\sigma),$$

where σ is a sequence, $f_k(\sigma)$ is the performance indicator k (normalised between 0 and 1, to simplify weights' calibration), τ is the number of the current generation, and w_k^τ is the weight, at iteration τ, of the kth indicator.

Downloaded by [Universita di Padova], [L. De Giovanni] at 09:58 14 May 2013

The dynamic definition of the fitness function (weights depend on τ) and the inclusion of local indicators are used to implement intensification and diversification phases. Intensification is intended to stress the search for individuals with better MOS, so that high MOS weights are used. Diversification searches for individuals with other good global or local features, by increasing the relative weights of TOS, IOS and/or NEW. We propose a three-phase 'intensification–diversification–intensification' search scheme. At the beginning (Phase I), we roughcast the initial population, whose MOS are still sufficiently different, by looking for individuals with a better objective function. Hence, a number of good individuals enter the population and their MOS tend to be included in a small range of values, meaning that many individuals have the same MOS. Thus, we use diversification (Phase II) to search for possibly different good features. Finally, a further intensification (Phase III) tries to translate the new promising individuals into improved-MOS pattern sequences. Different parameters are necessary to implement this search strategy, three sets of weights w_k^{I}, w_k^{II} and w_k^{III}, and two iteration thresholds, π^{I} and π^{II}, meaning that $w_k^{\tau} = w_k^{\mathrm{I}}$ for $\tau \leq \pi^{\mathrm{I}}$, $w_k^{\tau} = w_k^{\mathrm{II}}$ for $\pi^{\mathrm{I}} < \tau \leq \pi^{\mathrm{II}}$ and $w_k^{\tau} = w_k^{\mathrm{III}}$ for $\tau > \pi^{\mathrm{II}}$. Weights are initialised by Step (3) and changed by Step (17).

Note that all the state-of-the-art genetic approaches for MOSP (CGA, PTA, PMA, ECS and GRACS, see Section 2) implement some intensification/diversification scheme. CGA and PTA (de Oliveira and Lorena 2002, 2005) include different *schemata* into the population, that is partial solutions where the pattern to assign to some positions in the sequence is not determined: each schema represents a different region of the solutions space, to be exploited by constructive or training heuristics (intensification). PMA (Mendes and Linhares 2004) evolves multiple populations in parallel. In ECS and GRACS (de Oliveira and Lorena 2006), solutions generated by evolutionary algorithms or greedy randomised procedures are iteratively clustered, so that solutions in the same cluster are similar to each other (according to a 2-swap distance), and, at the same time, different clusters keep different characteristics; the search is then intensified in the most promising clusters by a 2-OPT local search. We remark also that different fitness functions are used, including just MOS (PMA, ECS, GRACS) or MOS and TOS (CGA and PTA). With respect to these metaheuristics, the intensification/diversification strategy adopted by AGA is simpler and easier to implement, as it is based on a dynamic fitness function including the new local performance indicators, so that little modification is required to the basic genetic schema, and no further algorithmic components. Nevertheless, as we will see in the computational section, results attest to its effectiveness.

Downloaded by [Universita di Padova], [L. De Giovanni] at 09:58 14 May 2013

4.4 *Evolution of the population*

The basic mechanism to evolve the population is the generation of new individuals from the current ones. In our algorithm, the cardinality of the population does not change from one iteration to the following one, and a mixed generational-replacement/steady-state strategy is adopted. The new population is divided into three subsets. A first *elite* subset of the current π_{E} fittest individuals is directly copied into the new population (Step (9)). The second subset of π_{X} individuals is generated by selection and crossover operators (inner cycle (5)–(8)). Selection (Step (6)) identifies two chromosomes from the current population, giving priority to the fittest: the first chromosome is randomly selected among the fittest π_{B} individuals and the second among all the current individuals. The two selected chromosomes are input to the *order crossover* operator (Step (7)), based on the classical one defined for the TSP, which guarantees the feasibility of each descendant (see the example in Figure 5 for a 10-pattern MOSP): two cut points are randomly chosen, dividing the chromosomes into three parts, and the two external sets of genes are copied unchanged from Parent 1 to Descendant 1 and from Parent 2 to Descendant 2; the remaining genes are filled-in in the order they appear in the other parent.

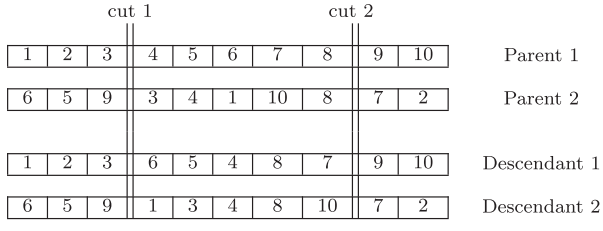


Figure 5. Order crossover.

Individuals in the last subset are randomly chosen and copied from the current population: one-half is chosen among the fittest π_B individuals and one-half among all the individuals (Step (9)). To avoid premature convergence towards homogeneous sequences, each individual undergoes a *mutation* with probability π_M^0: the mutation operator swaps a pair of randomly chosen genes (Step (11)). If no improving individual has been generated by the last generation (Step (15)), the mutation probability is increased up to a maximum value π_M^1. The parameter is reset to its default value π_M^0 as soon as an improving sequence is generated.

4.5 *Population refinement*

AGA implements a refinement operator based on a local search to efficiently explore the neighbourhood of the most promising individuals and replaces them with local optima (Step (13)). The local search starts from the initial chromosome (centre solution) and applies small perturbations that define a set of new solutions (*neighbourhood*). Then, a neighbour with better fitness than the centre solution becomes the centre of a new neighbourhood and the process is iterated until no solution in the current neighbourhood has a better fitness than the centre one, that is a local optimum has been reached, which replaces the initial chromosome. Neighbours are evaluated by the current fitness function, as refinement is intended to complement the genetic operators in finding fit individuals according to the criteria of the current search phase. As we have seen, the operator is also used during the generation of the initial population: in this case, MOS and TOS are lexicographically combined.

The refinement operator helps AGA to find the fittest solutions at the cost of additional running time. A good trade-off has to be achieved between the quality of the local optimum found at each refinement step, the overall quality of the solution provided by AGA and the additional computational effort. As a consequence, as in other state-of-the-art methods for MOSP, a simple 2-OPT neighbourhood is used, borrowed from the TSP literature (Lin and Kernighan 1973): given the centre sequence, neighbours are obtained by reversing each possible subsequence, as exemplified by Figure 6. In addition, to further speed-up the search, AGA introduces a limit π_L to the length of the swapping sequence and a limit π_D to the maximum number of descent moves per local search. Also, a first-improvement strategy has been implemented, which better trades-off search efficiency and final solution quality. Finally, we apply the refinement operator as little as possible: once every π_f^R generations to a restricted number of π_B^R fittest individuals. Calibration of the refinement parameters has been crucial to balance effectiveness and time efficiency.

4.6 *Stop criteria*

AGA iteratively evolves the initial population until one of the following stop criteria is met (AGA outer cycle (4)–(18)): a maximum number of π_G generations is reached, or a feasible solution is generated with MOS equal to the lower bound set by the pre-processing described in Section 4.2. AGA returns the sequence with the best MOS found during the search, after applying the necessary actions (Step (19)) to recover the pre-processing steps, that is completing with the removed items (see Section 4.2). Note that, as we have seen, in case the best MOS is equal to the lower bound, the solution found by AGA is optimal.

5. Numerical experiments

The performance of the proposed approach has been evaluated on several benchmarks from the literature (Smith and Gent 2005, Chu and Stuckey 2009), VLSI and the woodcutting industry (Hu and Chen 1990, SCOOP Team 2009) and on a new benchmark of large-size instances, for a total of over 6000 instances. The following results refer to a C++ implementation of AGA, run on a personal computer equipped with a 2.1 GHz Intel Core 2 processor and 2 GB RAM.

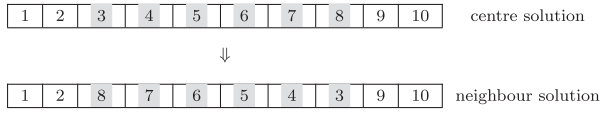


Figure 6. A sample 2-OPT neighbour obtained by substring ⟨3, 4, 5, 6, 7, 8⟩ reversal.

Downloaded by [Universita di Padova], [L. De Giovanni] at 09:58 14 May 2013

Downloaded by [Universita di Padova], [L. De Giovanni] at 09:58 14 May 2013

5.1 *Parameter setting*

As usual for genetic approaches, AGA performance depends on a number of parameters that must be calibrated, that is a general setting has to be found before its execution on a specific instance. This setting represents a suggestion to be used with any MOSP instance, and no further time-consuming parameter estimation is required. In this paper, we have followed the conventional hand-based approach (de Landgraaf *et al.* 2007): we have considered a subset of the available instances (to keep the time spent for calibration reasonable and obtain an as general as possible setting), run several experiments with different tentative settings, and analysed the results of these preliminary tests to fix the parameters. The proposed general setting is shown in Table 1 and in Table 2, which reports, for the three intensification–diversification–intensification phases, the weights of the adaptive fitness function and the duration in terms of total number of generations π_G.

The values of π_G, π_N, π_f^R, π_B^R, π_D, π_L and π_H have been fixed as small as possible, in particular for instances with a large number of patterns: larger values generally provide slightly better results, but the computational efficiency of AGA is reduced. Note that the value of some parameters depends on the number of patterns P, to take smaller instances into account: in this case, AGA needs fewer iterations and individuals to converge (parameters π_N and π_G), while more individuals can be generated by the initial heuristics and local search (parameter π_H) without degrading running times. For the remaining parameters, including the fitness function weights, they have been adjusted to guarantee AGA convergence while preserving population diversification, as discussed in the previous sections. Although more sophisticated techniques for parameter calibration may be applied and may obtain even better results, based, for example, on concepts from design of experiments, machine learning, artificial intelligence, data mining, etc. (Bagchi and Deb 1996, Eiben *et al.* 1999, Adenso-Diaz and Laguna 2006, Birattari 2009, Lessmann *et al.* 2011, among others), the proposed setting has been used for all the experiments described in the following and allows a comparison of AGA with state-of-the-art methods for MOSP, which is the scope of this work.

5.2 *Literature instances*

AGA has been compared with the state-of-the-art algorithm (CS) presented by Chu and Stuckey (2009). CS is an exact algorithm based on customer (item types) search, and combining branch-and-bound, nogood recording,

Table 1. AGA parameter summary.

Parameter		Number of patterns (P)	
Number of individuals per generation	π_N	$\min(10P, 500)$	
Number of generations	π_G	$\min(20P, 500)$	
Number of elite individuals	π_E	1	
Number of individual by crossover	π_X	$0.98\pi_N$	
Best individuals threshold	π_B	$0.40\pi_N$	
Mutation probability min, max	π_M^0, π_M^1	0.1, 0.2	
Refinement frequency	π_f^R	$\min(2P, 100)$	
Number of refined individuals	π_B^R	1	
		($P \leq 125$)	($P > 125$)
Local search depth	π_D	100	200
Local search width	π_L	10	5
Number of initial individuals by heuristics	π_H	$0.10\pi_N$	$0.05\pi_N$

Table 2. Parameters for search phases.

Phase (h)	w_{MOS}^h	w_{TOS}^h	w_{IOS}^h	w_{NEW}^h	π^h
I – Intensification	0.70	0.16	0.07	0.07	$0.35\pi_G$
II – Diversification	0.20	0.20	0.30	0.30	$0.50\pi_G$
III – Intensification	0.95	0.05	0.00	0.00	–

dominance rules, relaxation and an effective heuristic using a clever incomplete search. A summary of the results is reported in Table 3 (see De Giovanni *et al*. (2010) for details).

AGA is non-deterministic and was run 20 times on each instance from Smith and Gent (2005) and Hu and Chen (1990); the number of repetitions was reduced to five for the Chu and Stuckey (2009) benchmark due to the longer running times. Each row refers to average values for a set of instances or for meaningful individual instances. The first columns report, respectively, the benchmark (or the name of individual instances), the number of patterns and item types (minimum–maximum), and the number of instances in the set (one for individual instances). The second group of columns is devoted to AGA, showing the average percent error of the best solution with respect to the optimal solution (the best MOS for individual instances), the maximum error (in terms of number of additional stacks required) over all the instances, the standard deviation over the 20 or five runs (or, for individual instances, the success rate, that is the percentage of AGA runs finding the best known MOS), and the average running time (in seconds). The last columns report the performance of CS run using the same system configuration as AGA: the percent error (or the best MOS found by CS for individual instances), the time (in seconds) to find the best solution, the average and the maximum time to prove optimality. Values in column 'Err% (MOS)' are in bold if the optimal solution has been found for all the instances of the same set. AGA finds optimal or near-optimal MOS for the benchmarks from Smith and Gent (2005) and Hu and Chen (1990), and the error is limited to a few percent in the other cases. CS performs better, as it always finds an optimal solution in less time, with the meaningful exception of W4, a real instance from the VLSI industry with 141 patterns and 202 item types (Hu and Chen 1990). W4 is the largest literature MOSP instance and, in this case, AGA finds the best known MOS (27) in 95% of runs (28 otherwise) within one minute computation, while CS cannot provide a MOS better than 28 after three hours. Actually, CS finds the optimal solution for all instances up to 125 patterns and item types in almost no time, and proving optimality may require up to about 20 minutes. The results indicate that the computational time of CS may increase exponentially with size: from a few seconds for 100×100 instances, to minutes for 125×125 instances, and hours (failing to find the best known solution) for the 141×202 instance W4. Hence, we further investigated the performance for large-size instances, in particular those with a large number of item types.

Downloaded by [Universita di Padova], [L. De Giovanni] at 09:58 14 May 2013

Table 3. Computational results: literature instances.

Instances				AGA				CS			
Bench. (*name*)	P	I	N.	Err% (*MOS*)	Δ	dev% (*SR*%)	*T*(s)	Err% (*MOS*)	T_{best}(s)	T_{proof}(s)	T_{max}(s)
Harvey[a]	10–30	10–30	2130	0.0%	1	0.02	0.27	**0.0**%	0.00	0.00	0.00
Simonis[a]	10–30	10–40	3630	0.0%	1	0.01	0.06	**0.0**%	0.00	0.00	0.00
Wilson[a]	20–100	10–100	17	**0.0%**	0	0.00	0.30	**0.0**%	0.04	0.05	0.11
SP2[a]	50	50	1	***19***	0	*45*	10.85	***19***	0.06	0.08	n.a.
SP3[a]	75	75	1	***34***	0	*100*	36.67	***34***	0.31	1.00	n.a.
SP4[a]	100	100	1	***53***	0	*70*	81.02	***53***	0.75	25.89	n.a.
Random[b]	30	30	25	0.2%	1	0.00	1.93	**0.0**%	0.00	0.00	0.00
	40	40	25	0.4%	1	0.22	6.58	**0.0**%	0.00	0.00	0.00
	50	50	25	1.0%	1	0.22	17.26	**0.0**%	0.00	0.00	0.01
	50	100	25	1.1%	1	0.45	29.74	**0.0**%	0.02	1.63	8.03
	75	75	25	2.1%	2	0.56	54.58	**0.0**%	0.01	0.08	0.63
	100	50	25	0.7%	1	0.18	61.83	**0.0**%	0.00	0.00	0.03
	100	100	25	3.5%	4	0.96	118.56	**0.0**%	0.06	2.65	23.11
	125	125	25	4.6%	5	0.86	189.98	**0.0**%	2.70	122.60	1128.32
VLSI[c]	10–70	10–84	10	**0.0**%	0	0.00	3.17	**0.0**%	0.02	0.02	0.13
W4[c]	141	202	1	***27***	0	*100*	67.23	*28*	7.21	–	n.a.

Notes: [a]Smith and Gent (2005).
[b]Chu and Stuckey (2009).
[c]Hu and Chen (1990).
–, Optimal solution not found after 3 hours computation.
n.a., not applicable to individual instances.

5.3 *Large-size random instances*

Using the same random generator as Chu and Stuckey (2009), we have generated five instances for each triplet defined by the number of patterns chosen in the set {50, 100, 150}, the number of item types chosen in the set {200, 250}, and the density (average number of item types per pattern) chosen in the set {2, 4, 6, 8, 10}, for a total of 150 instances. We have compared AGA with CS and with the Parallel Memetic Algorithm (PMA, Mendes and Linhares 2004), one of the state-of-the-art genetic approaches for MOSP, described in Section 2. CS and PMA were run on the same configuration as AGA. In order to make an as fair as possible comparison, we have to take into account the non-deterministic nature of AGA and PMA, and the potentially long computational time of CS as an exact approach. Thus, we have run AGA five times for each instance and recorded the average computational time; then, CS and PMA have been run and stopped after the same computational time, storing the best (if not provably optimal) solution found so far (PMA has been repeated five times per instance). The results are synthesised in Table 4, showing a row with average results for each group of five instances with the same number of patterns (column 'P'), number of item types ('I') and density ('D').

The columns devoted to AGA and PMA report the best MOS over five runs, the average MOS, the percent standard deviation and the computational time in seconds. Columns devoted to CS are the value of the MOS and the number of instances with proven optimality within the time limit. Finally, the last column shows the number of instances where AGA provides a better–same–worse ('win–draw–lose') MOS than CS. AGA and PMA are comparable in terms of best solution: in 15 instance classes over 30, AGA has a better or the same value in column 'Best' and the difference is normally very small in the other cases. In terms of average solution, AGA provides better results in 20 classes over 30 (values in bold). Further, the standard deviation of AGA is always better, even

Table 4. Computational results: large random instances.

Instances			AGA MOS				PMA MOS			CS MOS		AGA vs. CS
P	I	D	Best	Avg	Dev%	T (s)	Best	Avg	Dev%	Best	Opt	Win–draw–lose
150	200	10	154.0	**154.7**	**0.8**	294.0	155.6	157.1	0.8	151.8	0	0–0–5
150	200	8	138.4	139.1	**0.6**	278.3	137.2	138.9	1.0	134.0	0	0–0–5
150	200	6	110.2	111.7	**1.5**	246.0	108.4	110.2	1.6	108.0	0	0–0–5
150	200	4	71.4	**72.2**	**1.4**	200.3	72.2	74.0	3.0	69.0	0	0–1–4
150	200	2	29.4	**30.1**	**2.9**	159.6	30.8	32.2	4.1	27.0	0	0–0–5
150	250	10	177.2	**177.8**	**0.5**	347.5	176.0	178.4	1.0	175.2	0	1–1–3
150	250	8	150.8	**152.2**	**0.7**	300.2	151.0	153.2	1.3	149.2	0	1–0–4
150	250	6	**110.4**	**110.9**	**0.5**	268.3	112.4	116.6	2.4	112.6	0	5–0–0
150	250	4	68.8	**70.1**	**1.8**	221.2	71.2	73.0	1.9	68.4	0	1–1–3
150	250	2	29.6	**30.6**	**4.1**	186.7	32.0	34.2	5.7	28.2	0	0–1–4
100	200	10	135.0	136.0	**0.7**	216.1	132.2	133.5	1.1	132.2	0	0–0–5
100	200	8	**110.0**	**110.7**	**0.9**	200.3	110.0	111.7	1.3	110.2	0	1–2–2
100	200	6	85.6	**86.4**	**1.2**	164.8	85.6	86.7	1.3	84.8	0	1–1–3
100	200	4	51.6	**52.1**	**1.4**	140.6	51.4	52.7	2.4	51.2	0	1–2–2
100	200	2	20.8	**21.6**	**5.0**	106.7	22.4	23.9	5.9	19.4	0	0–2–3
100	250	10	**143.6**	144.7	**0.8**	240.1	142.2	143.5	0.9	144.4	0	2–1–2
100	250	8	**112.4**	**113.1**	**0.9**	188.3	111.4	113.9	2.2	114.2	0	4–1–0
100	250	6	81.4	**82.4**	**1.1**	170.5	80.8	82.4	1.7	81.2	0	3–0–2
100	250	4	46.2	**46.5**	**1.0**	135.9	46.2	48.3	3.7	45.6	0	1–0–4
100	250	2	19.0	**20.1**	6.6	99.3	21.0	22.0	4.2	18.0	0	1–2–2
50	200	10	**79.0**	79.7	0.9	56.8	79.0	79.2	0.6	80.4	0	4–1–0
50	200	8	**63.0**	**63.1**	**0.3**	56.1	62.4	63.7	1.6	63.4	0	2–1–2
50	200	6	**49.0**	49.4	**0.5**	53.2	48.2	49.1	1.7	49.0	0	2–2–1
50	200	4	30.4	30.6	**1.1**	42.7	29.8	30.4	1.9	29.8	0	1–1–3
50	200	2	**13.4**	**13.7**	**1.3**	33.8	14.2	14.7	3.2	13.8	1	2–2–1
50	250	10	**84.0**	84.4	**0.5**	68.5	82.6	83.8	1.4	85.4	0	3–1–1
50	250	8	**66.4**	67.0	**0.9**	62.9	66.0	66.8	1.5	67.8	0	3–1–1
50	250	6	**45.2**	**45.3**	**0.4**	62.8	44.6	45.4	1.5	45.4	0	2–1–2
50	250	4	**26.6**	27.1	**2.2**	53.3	26.4	27.0	2.1	27.2	0	3–1–1
50	250	2	**15.0**	**15.2**	**3.6**	48.0	15.0	15.7	5.2	15.2	0	2–2–1

Downloaded by [Universita di Padova], [L. De Giovanni] at 09:58 14 May 2013

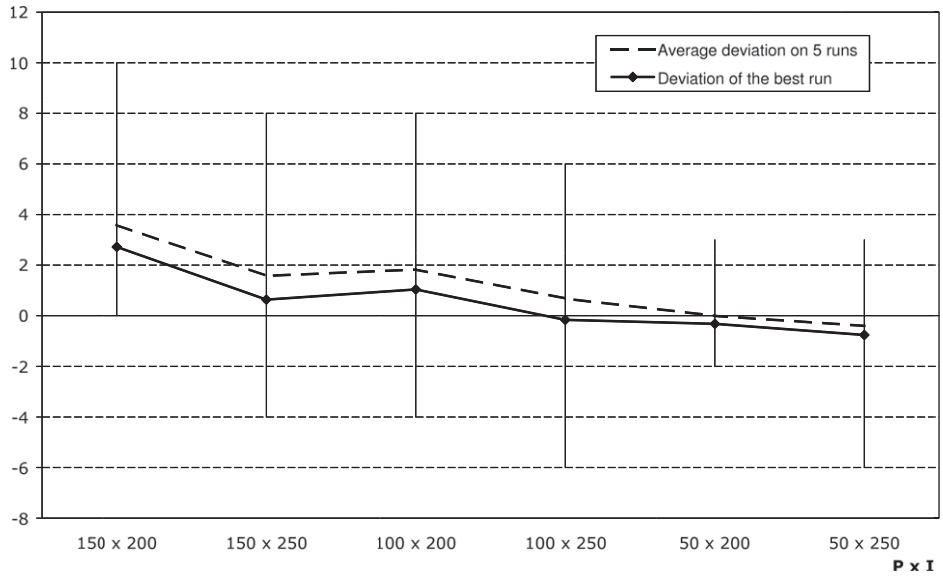


Figure 7. AGA–CS MOS deviation (lines are the average number of additional/saved stacks, bars are the min/max number of additional/saved stacks).

Downloaded by [Universita di Padova], [L. De Giovanni] at 09:58 14 May 2013

significantly, than that of PMA (values of AGA in bold), with the only exception of two classes: on average, the percent deviation of AGA is two-thirds that of PMA. Hence, AGA provides results comparable to PMA and is more reliable. Comparing AGA with CS, first we note that CS proves the optimality of just one of the 150 instances. Looking at MOS columns and 'win–draw–lose', we observe that CS still dominates AGA for 150×200 instances, whereas, for the remaining instances, no dominance relation exists, as, for a given size, there are always instances for which one of the approaches provides strictly better MOS than the other. In particular, for a number of groups, the average best MOS provided by AGA is the same or better than that found by CS (in these cases, the AGA best MOS is shown in bold), which is relevant for the 50×200 and 50×250 instances. Of course, we have to consider the non-deterministic nature of AGA with respect to the deterministic behaviour of CS. However, excluding lower densities, the average performance of AGA is normally very close to the best performance and the standard deviation is about 1% or less, meaning that AGA reliably provides better results.

Figure 7 gives an insight into the comparative performances of AGA and CS: it shows the trend of the absolute MOS deviation obtained as the difference between the best (solid line) or the average (dashed line) MOS over five runs of AGA, and the MOS found by CS: positive values are the additional stacks required by AGA, negative values the saved stacks. Each point refers to the average over 25 instances of the same size ('$P \times I$'), and bars show the maximum and the minimum deviation occurring in any of the five runs. The figure confirms that CS performs better for instances with 100 or more patterns and up to 200 item types, even if, for some specific instance, AGA finds remarkably better solutions, saving up to four stacks. As the number of item types increases and/or the number of patterns decreases, AGA performs better and better and becomes more and more competitive with respect to CS, which is not surprising, as CS is item-oriented and AGA is pattern-oriented. In particular, AGA provides savings of up to six stacks.

5.4 *Real VLSI instances*

In this section, we focus on VLSI instances with more than 40 patterns or item types, and we compare AGA with the state of the art on W4, the largest available real instance: PMA, the Constructive Genetic Algorithm (CGA, de Oliveira and Lorena 2002), the Evolutionary Clustering Search (ECS, de Oliveira and Lorena 2006) and the Greedy Randomized Adaptive Clustering Search (GRACS, de Oliveira and Lorena 2006), described in Section 2. Note that two sets of results are available for PMA: PMA-1, taken from Mendes and Linhares (2004) where PMA was run on a 366 MHz Pentium II processor, and PMA-2, where PMA was run on the same computer as AGA. Table 5 has one column for each instance, specifying name, size and best known MOS.

Table 5. Computational results: VLSI instances.

Instance	Name: / P × I: / Best MOS:	v4470 47 × 37 9	x0 48 × 40 11	W2 33 × 48 14	W3 70 × 84 18	W4 141 × 202 27
AGA	SR%	100	100	100	100	95
	Max err	0	0	0	0	1
	Time	5.3	6.6	0.0	18.9	67.2
	FC × 1000	573.2	745.8	0.0	990.0	1019.3
PMA – 1	SR%	60	100	100	90	20
	Max err	2	0	0	1	7
	Time[a]	10.0	30.0	30.0	90.0	2400.0
	FC × 1000	176.6	43.0	3.6	203.9	9428.6
PMA – 2	SR%	10	100	100	0	0
	Max err	1	0	0	5	3
	Time[b]	5.3	6.6	6.6	18.9	67.2
CGA	SR%	80	90	100	50	30
	Max err[c]	0	0	0	0	1
	Time[d]	66.5	75.6	18.5	306.3	5224.7
ECS	SR%	60	100	100	50	55
	FC × 1000	169.1	119.3	26.2	540.9	1695.9
GRACS	SR%	100	100	100	90	20
	FC × 1000	90.5	39.2	10.0	372.0	2357.5

Notes: [a]Time limit on a 366 MHz Pentium II processor.
[b]Time limit on the same system as AGA.
[c]Referred to the best five runs (as from de Oliveira and Lorena 2002).
[d]On a 266 MHz Pentium II processor.

With the exception of W4, the optimality of the remaining best known MOS has been proven by CS, which solves to optimality these VLSI instances in almost no time (see row 'VLSI' of Table 3, including results for v4470, x0, W2 and W3). All of the compared approaches are non-deterministic: AGA, PMA-2, ECS and GRACS results refer to 20 run per instance, CGA and PMA-1 to 10 runs (as from the information available in the related works). For each algorithm, rows report, where available, the success rate 'SR%' (percentage of runs finding the best known results), the maximum error over all the runs 'Max err', the average running time (in seconds), and the average number of (thousands) evaluation-function calls 'FC', as a further machine-independent measure of efficiency. AGA outperforms all other approaches in terms of success rate, as it always obtains the best known MOS, but for just one run on W4 (in this case the error is one). Other approaches have smaller success rates and, at least for PMA, larger errors. AGA also seems competitive in terms of efficiency, even if we remark that running times are related to different computational resources, and that CGA stops when the best known solution (used as input) is found, whereas AGA and PMA do not consider this speeding-up stop criterion. Also, the number of function calls is always limited. Note that, for W2, AGA has time and FC equal to zero, as the initial greedy heuristic finds a solution equal to the lower bound in a few milliseconds.

5.5 *Role of the adaptive fitness strategy*

AGA implements a heuristics-driven multi-start local search (LS) to provide part of the initial population, and a multi-phase search strategy is obtained by dynamically changing the weights of the composite fitness function to determine an intensification–diversification–intensification scheme (phases I–II–II). In order to verify the importance of this adaptive fitness strategy, we further analyse the instances where the initial local searches do not directly find the best AGA solution and genetic operators play some role. For these cases, we have examined the trend of MOS after phases I, II and III. Results are summarised in Table 6 and refer to the new benchmark of large random instances. Instances are aggregated by number of patterns (column 'P') and item types ('I'). Column 'n.i.' is the number of instances, over the 25 belonging to the same class, where genetic operators improve over the initial (deterministic) local searches. The remaining columns show the rate of runs finding the best AGA solution (Best rate (%)) after phases I, II and III. From column 'n.i.', we observe that the proposed genetic operators play an important

Downloaded by [Universita di Padova], [L. De Giovanni] at 09:58 14 May 2013

Table 6. Role of adaptive fitness: large random instances.

Instances			Best rate (%)		
P	I	n.i.	Ph. I	II	III
50	200	16	53.8	53.8	70.0
50	250	21	44.8	46.7	65.7
100	200	25	16.0	16.8	54.4
100	250	25	12.0	13.6	52.8
150	200	25	11.2	11.2	56.0
150	250	25	6.4	6.4	46.4
Total		*137*	*21.5*	*22.2*	*56.5*

role for almost all the instances. The evolution of the best rate during the three AGA phases confirms our expectations, for all the classes of instances. The average best rate after phase I is about 21% and remains almost the same at the end of the second diversification phase. The third phase provides a substantial improvement, with a boost ratio of two, on average, and up to eight for largest instances, proving the effectiveness of the proposed adaptive fitness function.

Downloaded by [Universita di Padova], [L. De Giovanni] at 09:58 14 May 2013

6. Conclusions

We have addressed the Minimisation of Open Stacks Problem (MOSP) and we have proposed a hybrid local search/genetic approach (AGA) characterised by a dynamic and composite fitness function. The key feature of AGA is the fitness function definition, able to attain two main goals: the first is to modify the search landscape shaped by MOS and drive the search towards promising regions, obtained by combining global and local performance indicators; the second is to alternate between search intensification and diversification, obtained by dynamically adapting components' weights. Computational results have been compared with the Customer Search (CS) proposed by Chu and Stuckey (2009), the state-of-the-art algorithm for MOSP. While the performance of AGA on small- and medium-sized instances is generally worse than CS, our approach has been shown to be competitive on large-sized instances, providing improvements, especially for instances with a large number of item types and a relatively small number of patterns. In fact, CS applies branch-and-bound to search in the space of item types, and the complexity of its components depends, again, on the number of item types, whereas AGA searches the pattern permutation space and, due to its metaheuristic nature, is less sensitive to an increase in instance size. Concerning the comparison with other state-of-the art metaheuristics, tests on the new benchmark and on real VLSI instances show that AGA is comparable in terms of the best solution found and outperforms other methods in terms of reliability.

From an algorithmic point of view, the main contribution of AGA is the definition of a new intensification/diversification strategy, and a new method for coping with MOS permutative redundancy, both based on search landscape modifications. This is obtained by introducing new local performance indicators in the fitness function definition and by dynamically adapting the fitness function parameters, instead of introducing further algorithmic components as in previous state-of-the-art genetic approaches. In so doing, AGA can be implemented as a rather plain adaptation of the general genetic schema. With respect to CS, AGA allows a more reliable control of the required computational resources while guaranteeing good-quality solutions, at least for instances with a large number of item types compared with the number of patterns. Also, the dynamic-programming-based approach proposed by CS is fully tailored for the MOS objective function, while AGA is more flexible and can be adapted to a different objective (or even to multiple objectives). In conclusion, AGA seems to be a good trade-off between quality of solutions, computational efficiency, plainness and flexibility of implementation.

The results for AGA may be extended by further research in different directions. A better static parameter setting may be obtained using tools for automatic parameter tuning. Also, dynamic parameter control may be introduced, using more sophisticated criteria than the proposed fixed one to automatically adapt fitness function weights to different search landscapes, for example by checking the convergence of the population and switching to diversification over a given threshold. Further, to speed up the search, some features may be borrowed from dynamic programming and customer-search-based approaches, such as the nogood concepts to exclude provably

bad regions from the search space, or the relaxing technique to identify and focus on core sub-problems. These issues are the subject of ongoing research.

Acknowledgements

This work was partially financed by the EU Commission, within the co-operative research project SCOOP (contract No. 032998) coordinated by Università Politecnica delle Marche, Ancona, Italy. The authors wish to thank Geoffrey Chu, Peter Stuckey, Alexandre Mendes and Alexandre Linhares for providing support in testing their algorithms.

References

Adenso-Diaz, B. and Laguna, M., 2006. Fine-tuning of algorithms using fractional experimental designs and local search. *Operations Research*, 54 (3), 99–114.

Andreatta, G., *et al*., 1989. Un problema min cutwidth generalizzato e sue applicazioni ad un FMS (A generalized min cutwidth problem and applications to FMS). *Atti delle giornate di lavoro AIRO*, 1–17, (in Italian).

Bagchi, T. and Deb, K., 1996. Calibration of GA parameters: the design of experiments approach. *Computer Science and Informatic*, 26 (3), 46–56.

Baptiste, P., 2005. Simple MIP formulations to minimize the maximum number of open stacks. *IJCAI'05 – Constraint modelling challenge*. Edinburgh, 9–13.

Becceneri, J.C., Yanasse, H.H., and Soma, N.Y., 2004. A method for solving the minimization of the maximum number of open stacks problem within a cutting process. *Computers & Operations Research*, 31 (14), 2315–2332.

Birattari, M., 2009. *Tuning metaheuristics: a machine learning perspective*. Vol. 197, Berlin: Springer.

Chu, G. and Stuckey, P.J., 2009. Minimizing the maximum number of open stacks by customer search. *Lecture Notes in Computer Science*, 5732, 242–257.

De Giovanni, L., Massi, G., and Pezzella, F., 2010. *Preliminary computational experiments with a genetic algorithm for the open stacks problem* [online], Available from: http://www.math.unipd.it/~luigi/manuscripts/201009_AGA_compPrel.pdf [Accessed 4 October 2011].

De Giovanni, L., *et al*., 2011. A heuristic and an exact method for pattern sequencing problems. *In*: *Proceedings of the VII ALIO/EURO workshop on applied combinatorial optimization*. Porto, Portugal.

de Landgraaf, W., Eiben, A., and Nannen, V., 2007. Parameter calibration using meta-algorithms. *In*: *Proceedings of CEC 2007 - IEEE congress on evolutionary computation*, 71–78.

de Oliveira, A.C.M. and Lorena, L.A.N., 2002. A constructive genetic algorithm for gate matrix layout problems. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 21 (8), 969–974.

de Oliveira, A.C.M. and Lorena, L.A.N., 2005. Population training heuristics. *Lecture Notes in Computer Science*, 3448, 166–176.

de Oliveira, A.C.M. and Lorena, L.A.N., 2006. Pattern sequencing problems by clustering search. *Lecture Notes in Computer Science*, 4140, 218–227.

Eiben, E., Hinterding, R., and Michalewicz, Z., 1999. Parameter control in evolutionary algorithms. *IEEE Transactions on Evolutionary Computation*, 3 (2), 124–141.

Faggioli, E. and Bentivoglio, C.A., 1998. Heuristic and exact methods for the cutting sequencing problem. *European Journal of Operational Research*, 110 (3), 564–575.

Fink, A. and Voß, S., 1999. Application of modern heuristic search methods to pattern sequencing problem. *Computers & Operations Research*, 26 (1), 17–34.

Garcia de la Banda, M. and Stuckey, P., 2007. Dynamic programming to minimize the maximum number of open stacks. *INFORMS Journal on Computing*, 19 (4), 607–617.

Hu, Y.H. and Chen, S.J., 1990. GM_Plan: a gate matrix layout algorithm based on artificial intelligence planning techniques. *IEEE Transactions on Computer-Aided Design*, 9 (8), 836–845.

Lessmann, S., Caserta, M., and Montalvo Arango, I., 2011. Tuning metaheuristics: a data mining based approach for particle swarm optimization. *Expert Systems with Applications*, 38 (10), 12826–12838.

Lin, S. and Kernighan, B.W., 1973. An effective heuristic algorithm for the traveling-salesman problem. *Operations Research*, 21 (2), 498–516.

Linhares, A. and Yanasse, H.H., 2002. Connections between cutting-pattern sequencing, VLSI design, and flexible machines. *Computers & Operations Research*, 29 (12), 1759–1772.

Lopes, I.C. and Valério de Carvalho, J.M., 2010. Using interval graphs in an order processing optimization problem. *In*: *Proceedings of the world congress on engineering WCE 2010*, Vol. III.

Mendes, A. and Linhares, A., 2004. A multiple-population evolutionary approach to gate matrix layout. *International Journal of System Science*, 35 (1), 13–23.

Downloaded by [Universita di Padova], [L. De Giovanni] at 09:58 14 May 2013

Miller, A., 2005. Improved lower bounds for solving the minimal open stacks problem. *IJCAI'05 fifth workshop on modelling and solving problems with constraints*. Edinburgh, 36–43.

SCOOP Team, 2009. *Sheet cutting and process optimization for furniture industry - SCOOP project* [online]. Available from: http://www.scoop-project.net [Accessed 4 October 2011].

Smith, B. and Gent, I., 2005. *IJCAI'05 – Constraint modelling challenge 2005*. Edinburgh.

Yanasse, H.H. and Senne, E.L.F., 2010. The minimization of open stacks problem: a review of some properties and their use in pre-processing operations. *European Journal of Operational Research*, 203 (3), 559–567.

Yuen, B.J., 1995. Improved heuristics for sequencing cutting patterns. *European Journal of Operational Research*, 87 (1), 57–64.

Yuen, B.J. and Richardson, K.V., 1995. Establishing the optimality of sequencing heuristics for cutting stock problems. *European Journal of Operational Research*, 84 (3), 590–598.

Downloaded by [Universita di Padova], [L. De Giovanni] at 09:58 14 May 2013